

ZOMBI2 performance

Scaling of species-tree and gene-family simulation

Performance analysis

generated for the ZOMBI2 project

2026-07-03

Abstract

We benchmark the two core operations of ZOMBI2 — simulating a species tree and evolving gene families along it — across five orders of magnitude in tree size, on a single laptop. Both are essentially linear in the number of tips. A **ten-million-tip species tree** is simulated in **≈ 3.2 min using 6.4 GB**; **one-million-tip** gene-family copy-number profiles are produced in **≈ 23 s using 2.9 GB**. The gene-family output was the one component that did not scale — a dense families $\times$ species matrix, quadratic in tip count — so we replaced it with a sparse (COO) representation, moving its ceiling from $\sim 100,000$ tips to several million. Replicate-level parallelism gives a $3.9\times$ speed-up on a 10-core (4 performance + 6 efficiency) chip. All raw timings, the code that produced them, and these figures are reproducible from the `performance_analysis/` workspace.

1 Setup and methodology

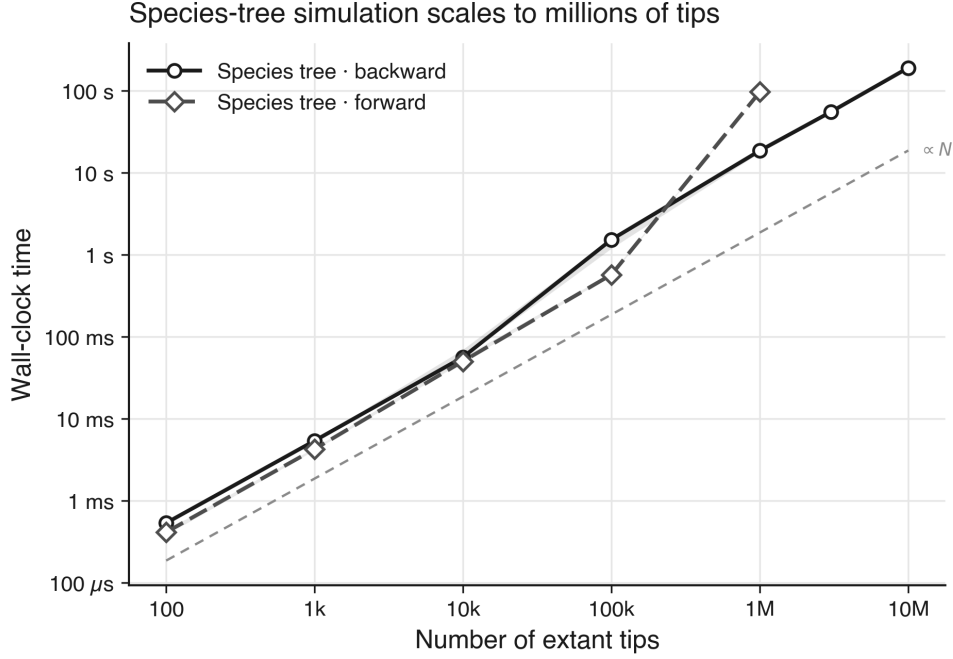
Workload. Every benchmark uses one fixed regime so the numbers stay comparable: a birth-death species model $\text{BirthDeath}(\lambda=1.0, \mu=0.3)$ of crown age 2.0, and gene families evolving under uniform per-copy rates ($D=0.2$, $T=0.1$, $L=0.25$, $O=0.5$; 20 founding families). Loss exceeds duplication, so family sizes never run away. The built-in model runs on the compiled `zombi2_core` (Rust) engine, which `simulate_genomes` selects automatically.

Measurement. Wall-clock time is taken with a monotonic clock (`time.perf_counter`) after one untimed warm-up; each point is repeated several times and we report the *median* with a shaded inter-quartile (25–75%) band. Species trees are built once per size and reused across the gene-family series, so those comparisons are a fair A/B on identical inputs; only the timed call sits inside the timer. Peak memory (resident set size) is measured in an *isolated subprocess* per size, the only way to get a clean per-size figure from a high-water-mark counter.

Machine. Apple Silicon, 10 cores (4 performance + 6 efficiency), 34 GB RAM, macOS; CPython 3.12, ZOMBI2 0.2.0.dev0. Absolute numbers are machine-specific; the *scaling* is not.

2 Species-tree simulation scales to tens of millions of tips

Simulating the species tree is $O(N)$ in the number of tips and featherweight (Figure 1). The backward reconstructed sampler — the default, which conditions on N extant tips — takes 55 ms at 10,000 tips, 12.4 s at 1,000,000, and 189.8 s at 10,000,000. The forward complete-tree sampler is comparable up to $\sim 100,000$ tips but grows heavier beyond, because it materialises every extinct lineage too (a 10^6 -tip *extant* tree carries $\approx 1.4 \times 10^6$ total nodes).



zombi2 0.2.0.dev0 · @1d07a37 · Rust · arm · Python 3.12.2 · 2026-07-03

Figure 1. Wall-clock time to simulate a species tree versus number of extant tips (log-log). Both samplers track the $\propto N$ guide; the backward sampler reaches 10,000,000 tips. Median of repeated runs; band is the inter-quartile range.

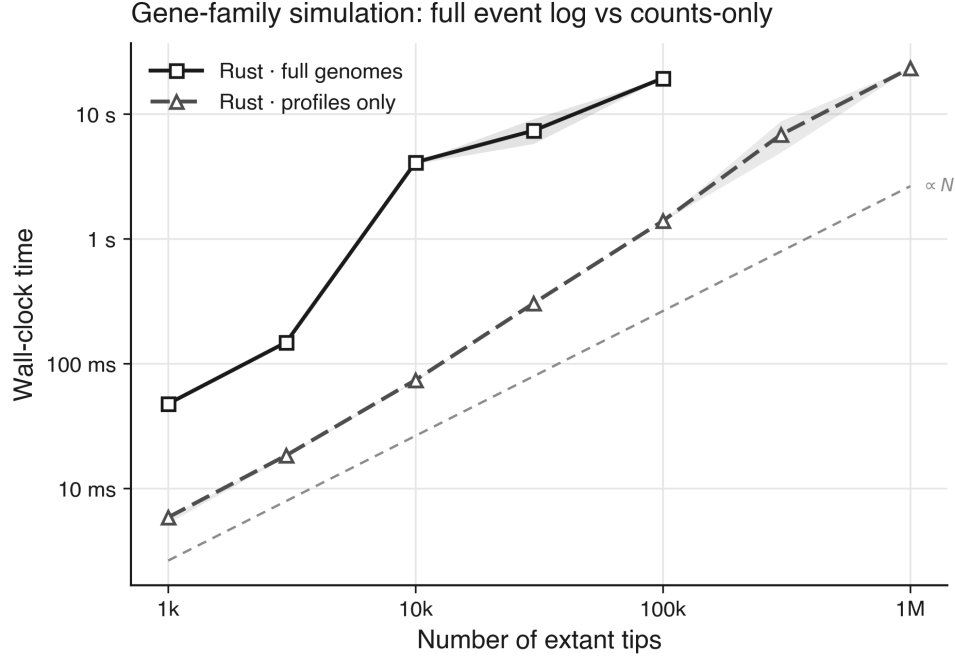
3 Gene-family simulation: full log vs. counts-only

Evolving D/T/L/O gene families along a fixed tree offers two outputs (Figure 2). The *full event log* records every event as an object, so its cost tracks the event count and it is practical to $\sim 100,000$ tips (19.2s, 3,052,812 events). The *counts-only* profile — the presence/absence dataset used for approximate Bayesian computation and downstream analyses — is far cheaper and, after the sparse-output change (§4), scales linearly into the millions: 71 ms at 10,000 tips, 1.3 s at 100,000, and 23 s at 1,000,000 tips (318,930 families).

4 Memory footprint and the sparse-output change

Peak memory (Figure 3) tells the “how big can we go” story. The species tree is the lightest, roughly 0.65 GB per million tips (76 MB at 10,000, 716 MB at 1,000,000, 6.4 GB at 10,000,000).

The copy-number *profile* was originally a dense families $\times$ species array. Because the number of families grows in proportion to the number of species, that array is $O(N^2)$ — it would demand ≈ 25 GB of address space already at 100,000 tips and ≈ 8 TB at 1,000,000, a hard wall well before the simulation itself struggled. We replaced it with a **sparse (COO) representation** storing only the present cells, which is $O(N)$: the profile now costs 422 MB at 100,000 tips, 3.2 GB at 1,000,000, and 7.4 GB at 3,000,000 (958,351 families) — sizes the dense form could never reach. All summaries (presence, genome sizes, frequency spectrum) and the on-disk output are computed directly from the sparse arrays, so nothing densifies. The full event log remains the heaviest path, as it must materialise one record per event.



zombi2 0.2.0.dev0 · @1d07a37 · Rust · arm · Python 3.12.2 · 2026-07-03

Figure 2. Gene-family simulation versus tip count (log-log). The counts-only profile runs straight along the $\propto N$ guide to 1,000,000 tips; the full event log is heavier and caps near 100,000 tips.

5 Replicate-level parallelism

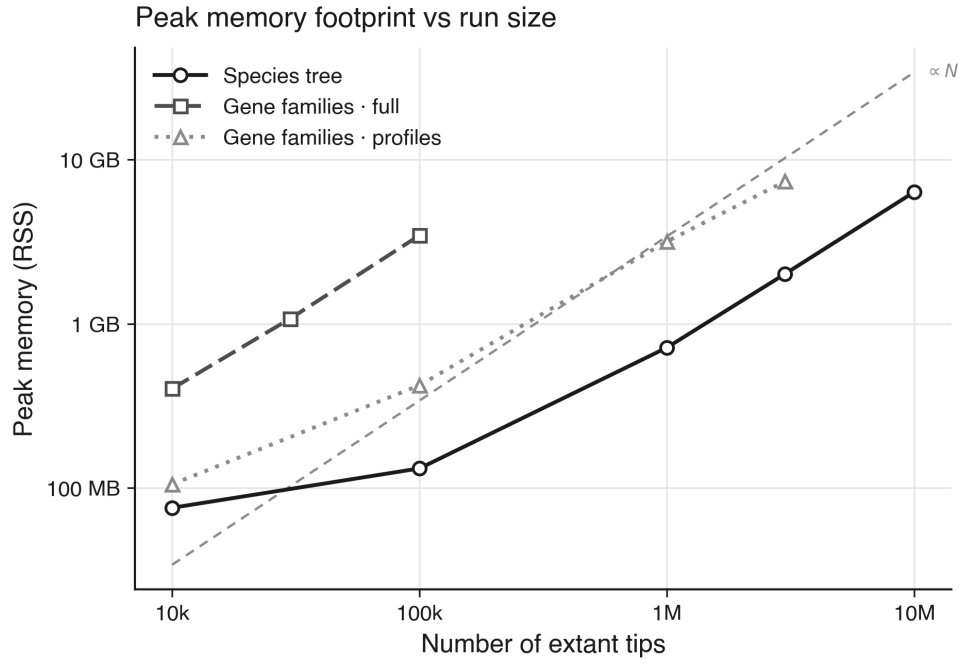
Independent replicates are embarrassingly parallel. Running 20 independent full simulations of 10,000-tip trees through a process pool (Figure 4) drops the wall-clock from 21.4s on one worker to 5.4s on ten — a $3.9\times$ speed-up. The curve bends away from the ideal beyond four workers: this chip has only four performance cores (the other six are slower efficiency cores), and on macOS each worker re-imports the library under the `spawn` start method. On a homogeneous, `fork`-based cluster (e.g. Linux/SLURM) the same workload scales closer to ideal.

6 Summary

Table 1. Headline results at representative scales (median wall-clock; peak RSS from the isolated-subprocess memory runs).

Task	Tips N	Time	Unit	Peak memory
Species tree (backward)	1,000,000	12.4	s	716 MB
Species tree (backward)	10,000,000	189.8	s	6.4 GB
Gene families, full log	100,000	19.2	s	3.5 GB
Gene families, profiles	1,000,000	23.0	s	2.9 GB
Gene families, profiles	3,000,000	104.6	s	7.4 GB

Bottom line. Both core operations are linear in tip count and run at million-tip scale on a laptop; the species tree reaches ten million. The only super-linear component was the dense profile output, now sparse and linear. The remaining heavy path is the full per-event log, which is inherent to recording every event; where only the presence/absence profile is needed, the

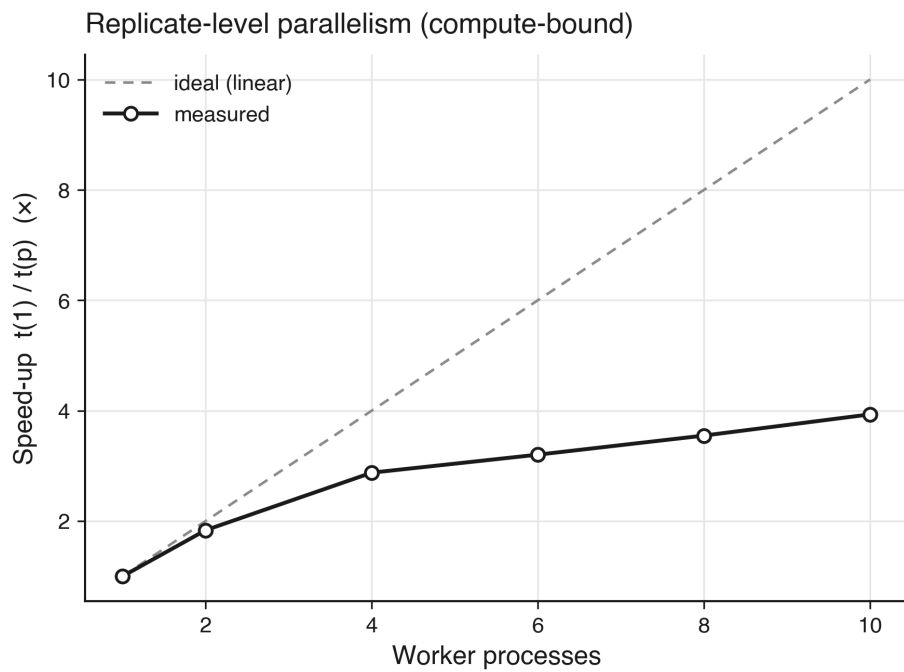


zombi2 0.2.0.dev0 · @1d07a37 · Rust · arm · Python 3.12.2 · 2026-07-03

Figure 3. Peak resident memory versus tip count (isolated subprocess per size). The species tree and the sparse profiles both scale linearly into the millions; the full event log is the memory-heaviest path.

counts-only path is the one to use at scale. An overview of all four measurements is shown in Figure 5.

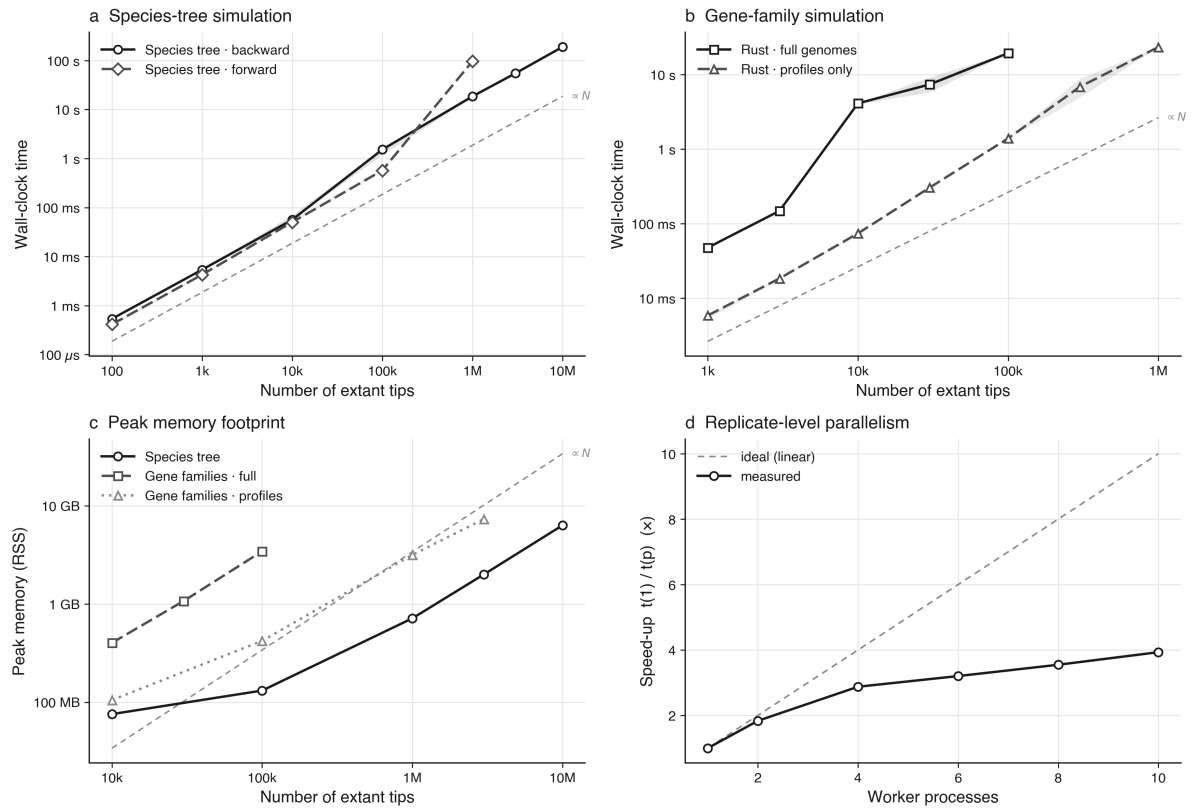
Reproducible from `performance_analysis/`: `python run.py` writes raw timings to `results/*.json`; `python plot.py` renders the figures. Each result file embeds its own provenance (git commit, interpreter, CPU).



zombi2 0.2.0.dev0 · @1d07a37 · Rust · arm · Python 3.12.2 · 2026-07-03

Figure 4. Speed-up of a fixed batch of independent simulations versus worker processes, against the ideal-linear reference. The plateau past four workers reflects the 4-performance/6-efficiency-core split and `spawn` overhead.

ZOMBI2 performance



zombi2 0.2.0.dev0 · ©1007a37 · Rust · arm · Python 3.12.2 · 2025-07-03

Figure 5. Overview: (a) species-tree scaling, (b) gene-family scaling, (c) peak-memory footprint, (d) replicate-level parallelism.